

Practical: Parallel Breadth First Search using OpenMP

Laboratory Practice V – Computer Engineering

1 Aim

Design and implement Parallel Breadth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS.

1.1 Objective

Students should be able to perform Parallel Breadth First Search based on existing algorithms using OpenMP.

2 Theory

2.1 What is BFS?

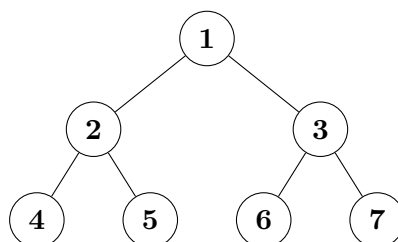
BFS (Breadth-First Search) is a graph/tree traversal algorithm that explores nodes **level by level**. Starting from the root, it visits all neighbors at the current depth before moving to the next depth level. It uses a **Queue** (FIFO) data structure.

2.1.1 BFS Algorithm Steps

1. Insert the root node into the Queue.
2. While the Queue is not empty:
 - (a) Dequeue the front node.
 - (b) Visit/Print the node.
 - (c) Enqueue all its children (left, then right).
3. Queue empty $\Rightarrow$ BFS complete.

2.2 BFS Example

Consider the following binary tree:



Queue Simulation:

Step	Queue (Front→Back)	Output
Start	[1]	—
Pop 1	[2, 3]	1
Pop 2	[3, 4, 5]	1, 2
Pop 3	[4, 5, 6, 7]	1, 2, 3
Pop 4	[5, 6, 7]	1, 2, 3, 4
Pop 5	[6, 7]	1, 2, 3, 4, 5
Pop 6	[7]	1, 2, 3, 4, 5, 6
Pop 7	[]	1, 2, 3, 4, 5, 6, 7

BFS Output: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

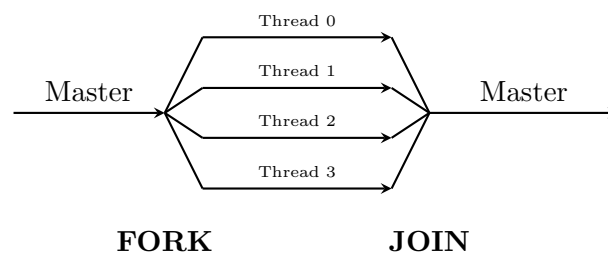
2.3 Applications of BFS

- Shortest path in unweighted graphs (GPS, Maps)
- Social network friend suggestions
- Web crawling (Search engines)
- Network broadcasting and packet routing
- Puzzle solving (maze, chess)

2.4 What is OpenMP?

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C, C++, and Fortran. It uses compiler directives (`#pragma`) to parallelize code.

2.4.1 Fork-Join Model



2.4.2 Important OpenMP Directives

Directive	Purpose
<code>#pragma omp parallel</code>	Creates multiple threads
<code>#pragma omp parallel for</code>	Divides loop iterations among threads
<code>#pragma omp critical</code>	Only one thread enters at a time (mutex)
<code>#pragma omp single</code>	Only one thread executes the block
<code>#pragma omp barrier</code>	Synchronization — wait for all threads
<code>#pragma omp task</code>	Creates an independent task

2.5 How Parallel BFS Works

1. Start with root in the queue.
2. At each level, count the number of nodes (qSize).
3. Use `#pragma omp parallel for` to divide those nodes among threads.
4. Each thread pops a node, prints it, and pushes its children.
5. `#pragma omp critical` protects queue operations (shared resource).
6. Repeat until queue is empty.

3 Complete Binary Tree vs BST

The code builds a **Complete Binary Tree**, not a BST.

Feature	Complete Binary Tree	BST
Insertion Rule	First empty slot (L→R)	Left < Root < Right
Value Comparison	No	Yes
Balanced?	Always	Not guaranteed
Parallelism	Better (max nodes/level)	Can be skewed

Why Complete Binary Tree is better for Parallel BFS:

- Every level has maximum possible nodes $\Rightarrow$ more threads work in parallel.
- BST can become skewed (e.g., sorted input: 1,2,3,4,5) $\Rightarrow$ one node per level $\Rightarrow$ no parallelism benefit.

4 Code

Listing 1: Parallel BFS using OpenMP

```

1 #include <iostream>
2 #include <stdlib.h>
3 #include <queue>
4 using namespace std;
5
6 class node {
7 public:
8     node *left, *right;
9     int data;
10 };
11
12 // Insert node into Complete Binary Tree (level-order)
13 node *insert(node *root, int data) {
14     if(!root) {
15         root = new node;
16         root->left = NULL;
17         root->right = NULL;
18         root->data = data;
19         return root;

```

```

20     }
21     queue<node *> q;
22     q.push(root);
23     while(!q.empty()) {
24         node *temp = q.front();
25         q.pop();
26         if(temp->left == NULL) {
27             temp->left = new node;
28             temp->left->left = NULL;
29             temp->left->right = NULL;
30             temp->left->data = data;
31             return root;
32         } else {
33             q.push(temp->left);
34         }
35         if(temp->right == NULL) {
36             temp->right = new node;
37             temp->right->left = NULL;
38             temp->right->right = NULL;
39             temp->right->data = data;
40             return root;
41         } else {
42             q.push(temp->right);
43         }
44     }
45 }
46
47 // Parallel BFS Traversal using OpenMP
48 void bfs(node *head) {
49     queue<node*> q;
50     q.push(head);
51     int qSize;
52     while (!q.empty()) {
53         qSize = q.size();
54         #pragma omp parallel for          // Parallel threads
55         for (int i = 0; i < qSize; i++) {
56             node* currNode;
57             #pragma omp critical          // Safe queue pop
58             {
59                 currNode = q.front();
60                 q.pop();
61                 cout << "\t" << currNode->data;
62             }
63             #pragma omp critical          // Safe children push
64             {
65                 if(currNode->left)
66                     q.push(currNode->left);
67                 if(currNode->right)
68                     q.push(currNode->right);
69             }
70         }
71     }
72 }

```

```
73
74 int main() {
75     node *root = NULL;
76     int data;
77     char ans;
78     do {
79         cout << "\n enter data=>";
80         cin >> data;
81         root = insert(root, data);
82         cout << "do you want insert one more node?";
83         cin >> ans;
84     } while(ans == 'y' || ans == 'Y');
85     bfs(root);
86     return 0;
87 }
```

5 Code Explanation

1. **Node Class:** Each node stores **data**, **left** pointer, and **right** pointer.
2. **insert() Function:** Uses a queue to find the first empty position in level-order and inserts the new node there, building a Complete Binary Tree.
3. **bfs() Function (Parallel):**
 - Pushes root into queue.
 - At each level, counts nodes (`qSize`).
 - `#pragma omp parallel for` — divides loop iterations among threads.
 - `#pragma omp critical (1st)` — safely pops node from queue and prints it.
 - `#pragma omp critical (2nd)` — safely pushes children into queue.
4. **main():** Takes user input in a do-while loop, builds tree, then calls `bfs()`.

5.1 Why Critical Section is Needed

Queue is a **shared resource**. Without critical sections:

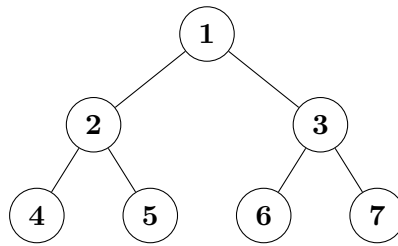
- Thread 0 calls `q.front()` → gets Node A
- Thread 1 calls `q.front()` *simultaneously* → also gets Node A (WRONG!)
- **Race Condition** ⇒ data corruption, crash

With `#pragma omp critical`, only one thread accesses the queue at a time.

6 Output

6.1 Example 1

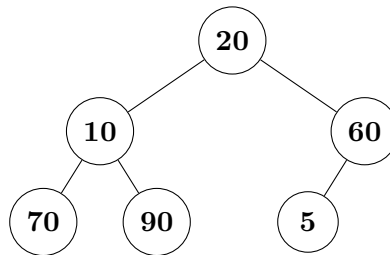
Input: 1, 2, 3, 4, 5, 6, 7



BFS Output: 1 2 3 4 5 6 7

6.2 Example 2

Input: 20, 10, 60, 70, 90, 5



- Level 0: 20
- Level 1: 10, 60
- Level 2: 70, 90, 5

BFS Output: 20 10 60 70 90 5

7 BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack
Traversal Order	Level-wise	Depth-wise
Shortest Path	Yes (unweighted)	Not guaranteed
Memory Usage	More (stores entire level)	Less
Parallelism	Easier (level-wise split)	Harder (task-based)

8 Time Complexity

	Sequential BFS	Parallel BFS
Time	$O(V + E)$	$O\left(\frac{V+E}{P}\right)$ ideally
Space	$O(V)$	$O(V)$

Where V = Vertices, E = Edges, P = Number of Threads.

9 Compilation

Listing 2: Compile and Run

```
g++ -fopenmp BFS.cpp -o BFS
./BFS
```

Note: The `-fopenmp` flag is mandatory. Without it, OpenMP directives are ignored and code runs sequentially.

10 Conclusion

In this practical, we implemented Parallel Breadth First Search on a Binary Tree using OpenMP. The `#pragma omp parallel` directive was used to process nodes at each level in parallel across multiple threads, while `#pragma omp critical` sections ensured thread-safe access to the shared queue. This demonstrates how parallelism can significantly improve BFS performance on multi-core processors compared to the sequential approach.